逐阶降阶揭示高阶相互作用作用

对《Unveiling the role of higher-order interactions via stepwise reduction》的深度研究报告

执行摘要

本文研究的核心问题是：

高阶相互作用是否必须显式建模，还是可以在尽量少损失信息的前提下，用较低阶结构来近似？

作者以超边预测（hyperedge prediction）为统一量化入口，提出 `n-reduced graph` 方法：保留所有阶数不超过 `n` 的超边，同时将所有高于 `n` 阶的超边分解为全部 `n` 元子集超边。这样可以在不改变原始节点集合的前提下，逐步保留更多高阶信息，并衡量高阶结构对预测任务的真实贡献。

在 12 个真实超图数据集上，作者发现：

1. 随着 `n` 增大，AUC 整体上升；
2. 但边际收益递减；
3. 不同网络对高阶信息的依赖差异明显。

同时，作者将 `n-reduced` 与 Yoon 等人在 WWW 2020 提出的 `n-projected graph` 进行了系统对比。在统一特征和逻辑回归设置下，`n-reduced` 明显更优。论文给出的代表性结果是：

- `n-projected`：最佳 AUC 约 `0.74`，平均约 `0.62`
- `n-reduced`：最佳 AUC 约 `0.94`，平均约 `0.83`

因此，这篇论文的价值不只是提出了一个新表示方法，更重要的是提供了一个可操作的“高阶信息贡献分析工具”，可以回答：

- 高阶结构到底有没有用；
- 要保留到几阶才够；
- 在不同任务中，高阶信息究竟贡献了多少。

---

1. 论文基本信息

- 题目：**Unveiling the role of higher-order interactions via stepwise reduction**
- 作者：**Junhao Bian, Tao Zhou, Yilin Bi**
- 期刊：**Communications Physics**
- 年份：**2025**
- DOI：10.1038/s42005-025-02157-3
- 预印本：`arXiv:2411.05685`
- 代码仓库：jackyjh/n-reduction-graph

1.1 获取与版本说明

本报告以你上传的 PDF 与期刊官网正式版本为准。

可用来源：

- 期刊正式版：Communications Physics
- DOI 页面：https://doi.org/10.1038/s42005-025-02157-3

- 代码与数据仓库：https://github.com/jackyjh/n-reduction-graph

如果后续需要追踪被引情况，可使用：

- Nature 页面中的 citation 指标
- OpenCitations
- Google Scholar / Scopus / Web of Science

OpenCitations 接口示例：

```
https://api.opencitations.net/index/v2/citations/doi%3A10.1038/s42005-025-02157-3
https://api.opencitations.net/index/v2/citation-count/doi%3A10.1038/s42005-025-02157-3
```

---

2. 一句话概括论文

这篇论文最核心的贡献，是提出了 `n-reduced graph` 这一逐阶降阶框架，使研究者可以在尽量保留结构信息的前提下，定量分析高阶相互作用对超边预测任务的真实贡献。

作者的最终结论不是“高阶一定重要”，而是：

高阶相互作用在不同网络中的作用差异很大，在一些系统里非常关键，在另一些系统里增益有限。

---

3. 论文所处的研究脉络

这篇论文位于两条研究脉络的交汇处：

1. 高阶网络/超图建模
2. 超边预测 / 高阶链接预测

3.1 高阶网络总体背景

- Battiston et al., **Networks beyond pairwise interactions**, Physics Reports, 2020
  高阶网络领域最重要的综述之一，系统讨论超图、单纯复形以及相关动力学。

3.2 高阶链接/超边预测的重要工作

- Benson et al., **Simplicial closure and higher-order link prediction**, PNAS, 2018
  将高阶链接预测系统化，是本文选择“超边预测”作为统一任务的重要背景。
- Zhang et al., **Beyond Link Prediction: Predicting Hyperlinks in Adjacency Space**, AAAI, 2018
  较早明确将 hyperlink prediction 正式化。

3.3 与本文最直接相关的方法

- Yoon et al., **How Much and When Do We Need Higher-order Information in Hypergraphs?**, WWW 2020
  提出 `n-projected graph`，是本文最重要的直接比较对象。

3.4 本文对照的其他预测方法

作者在补充材料中还使用了多种方法进行稳健性验证，包括：

- HPRA
- NHP
- NHNE

这说明本文的结论不是只在一个非常狭窄的 baseline 上成立。

---

4. 研究背景与问题定义

传统图模型只能表达两两关系，但许多现实系统天然具有高阶关系：

- 一篇论文由多个作者共同完成；
- 一封邮件可能同时涉及多个参与者；
- 一个生化反应往往涉及多个底物和酶；
- 生态系统中两个物种之间的关系可能受第三个物种调节。

这时超图更自然，因为一条超边可以同时连接多个节点。

但高阶建模带来一个核心争议：

这些高阶信息到底值不值得保留？

更具体地说：

1. 哪些高阶信息是必须保留的？
2. 哪些高阶信息可以被低阶结构近似？
3. 如果只保留到某一阶，会损失多少有效信息？

为回答这一问题，作者选择超边预测作为量化任务。

4.1 什么是超边预测

给定观测到的超图结构，判断一个尚未观测到的节点集合是否是真实超边。

本质上，这是一个二分类任务：

- 正样本：真实存在的超边
- 负样本：不存在的节点集合

作者用 AUC 作为统一衡量指标，从而把“高阶信息是否有用”转化为“预测性能是否提升”。

---

5. 四种处理高阶超边的方法

为了理解本文方法，先把它放到更大的方法背景中看。

5.1 团扩展（Clique Expansion）

思想：

> 把一条高阶超边中的所有节点两两相连，得到普通图中的完全子图。

例如，对 `{A,B,C,D}` 做团扩展，得到：

- `AB`
- `AC`
- `AD`
- `BC`
- `BD`
- `CD`

优点：

- 简单
- 可直接使用普通图算法

缺点：

- 一个整体群体交互被打散成了很多两两关系
- 高阶结构的整体性消失

5.2 星拓展（Star Expansion）

思想：

> 为每条超边引入一个新的“超边节点”，然后让它与超边中的所有原始节点相连。

例如，对 `{A,B,C,D}` 做星拓展，得到：

- `e1-A`
- `e1-B`
- `e1-C`
- `e1-D`

优点：

- 保留了“这些节点属于同一条超边”的信息

缺点：

- 结构变成了二部图
- 分析对象从“原始节点之间的高阶交互”变成了“节点-超边节点”关系

5.3 n-projected graph

这是 Yoon 等人提出、本文重点比较的方法。

思想：

> 把由 `n-1` 个原始节点组成的集合作为新图中的节点。

然后，如果两个这样的 `(n-1)` 集合：

- 只差一个元素；
- 并集大小正好是 `n`；
- 且这个并集被某条原始超边包含；

则在这两个“复合节点”之间连边。

举例：

若原始超边是 `{A,B,C,D}`，当 `n=3` 时，`n-projected` 图中的节点会是所有 2 元集合：

- `AB`
- `AC`
- `AD`
- `BC`
- `BD`
- `CD`

这些集合节点之间再按规则连边。

优点：

- 比普通团扩展保留了更多组合信息

缺点：

- 新图中的节点不再是原始实体，而是“节点集合”
- 节点复杂度和异质性迅速增加
- 不适合精确回答“高于某一阶的信息到底还有没有额外贡献”

5.4 n-reduced graph

这是本文提出的方法。

思想：

> 保留所有阶数不超过 `n` 的超边，并把所有高于 `n` 阶的超边分解为所有可能的 `n` 元子超边。

例如，对 `{A,B,C,D}`，当 `n=3` 时，得到：

- `ABC`
- `ABD`
- `ACD`
- `BCD`

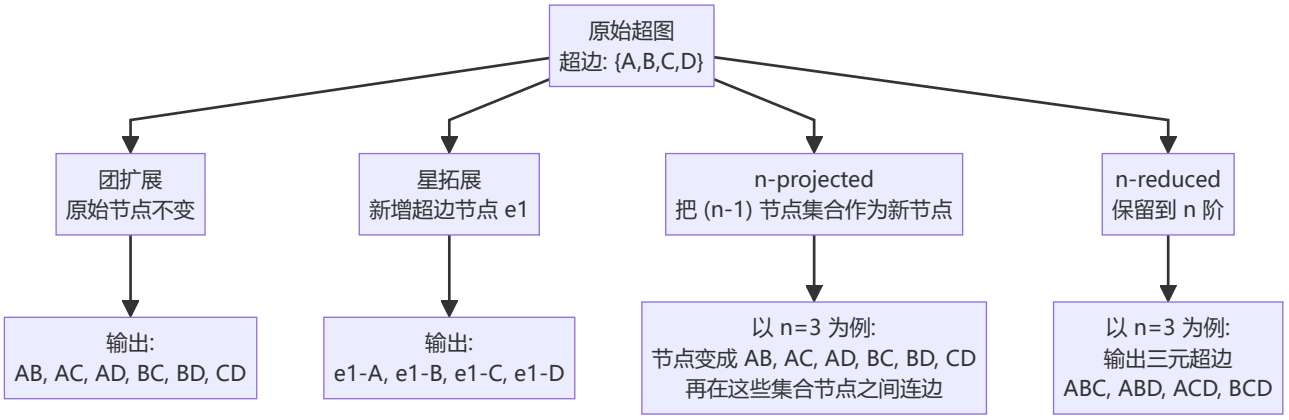
优点：

1. 原始节点集合保持不变
2. 不高于 `n` 阶的真实超边原样保留
3. 更高阶超边不是被删除，也不是彻底压成 pairwise，而是逐步降到 `n` 阶
4. 特别适合分析“保留到几阶才够”

5.5 四种方法的对照表

方法	节点是什么	输出结构	高阶信息如何处理	主要问题
团扩展	原始节点	两两边	全部压成 pairwise	群体整体性丢失
星拓展	原始节点 + 超边节点	二部图边	通过新节点保存超边身份	改变原始对象类型
`n-projected`	`(n-1)` 节点集合	复合节点图	编码到集合节点之间的边	节点复杂度高
`n-reduced`	原始节点	不超过 `n` 阶的超边	高阶超边分解成 `n` 阶超边	仍可能有组合增长

5.6 并列示意图



5.7 PPT 可直接使用的示意图文案

页面标题建议：

高阶超边的四种处理方式对比

页面副标题建议：

以原始超边 `{A,B,C,D}` 为例，对比不同方法如何处理高阶信息

四栏标题建议：

1. `Clique Expansion`
2. `Star Expansion`
3. `n-projected`
4. `n-reduced`

页脚总结语建议：

团扩展最简单，星拓展保留超边身份，n-projected 强调组合映射，n-reduced 最适合逐步分析高阶信息贡献。

---

6. 为什么不能直接删除高阶超边

如果想研究“5 阶及以上超边是否重要”，一个最粗糙的方法是：

- 直接删掉所有 5 阶及以上超边
- 再比较任务性能

但这样做的问题很大，因为你删掉的是“全部信息”。

如果性能下降，你无法区分：

1. 是高阶整体结构本身重要；
2. 还是因为删得太狠，把其中本来可以保留的低阶线索也一起删掉了。

因此，本文强调：

不要粗暴删除，而要逐步降阶。

---

7. n-reduced 的核心定义与直观理解

作者定义：

- 如果超边阶数 `k ≤ n`，原样保留；
- 如果超边阶数 `k > n`，则生成其所有 `n` 元子集超边。

因此，`n` 可以理解为一个“高阶信息保留深度”的控制旋钮：

- `n = 2`：接近 pairwise 层面
- `n = 3`：显式保留三元关系
- `n = 4`：进一步保留四元关系
- `n = max(k)`：接近原始超图

本文的关键思想是：

随着 `n` 增大，模型保留的信息越来越接近原始超图，因此不同 `n` 下的任务性能差异，可以被解释为不同阶高阶信息的贡献差异。

---

8. 一个完整的简单实验

为了帮助理解论文实验逻辑，这里构造一个最小可解释的玩具实验。

8.1 原始超图

节点集合：

`V = {A, B, C, D, E, F}`

原始超边：

- `{A,B,C}`
- `{A,B,D}`
- `{A,B,C,D}`
- `{B,C,E}`
- `{C,D,F}`

8.2 两种降阶方式

当 `n = 2`

得到 pairwise 结构：

`AB, AC, BC, AD, BD, CD, BE, CE, CF, DF`

当 `n = 3`

得到三元超边结构：

- `ABC`
- `ABD`
- `ACD`
- `BCD`
- `BCE`
- `CDF`

8.3 候选三元集合

选择 5 个候选集合：

- `{A,C,D}`
- `{A,C,E}`
- `{A,D,F}`
- `{A,E,F}`
- `{B,D,E}`

8.4 特征对比结果

候选集合	n=2 特征 [GM, HM, AM, CN, JC, AA]	n=3 特征 [GM, HM, AM, CN, JC, AA]
{A,C,D}	[1.0, 1.0, 1.0, 1, 0.167, 0.721]	[2.289, 2.25, 2.333, 1, 0.167, 0.721]
{A,C,E}	[1.0, 1.0, 1.0, 1, 0.167, 0.721]	[1.414, 1.333, 1.5, 1, 0.167, 0.721]
{A,D,F}	[1.0, 1.0, 1.0, 1, 0.2, 0.621]	[1.414, 1.333, 1.5, 1, 0.2, 0.621]
{A,E,F}	[0, 0, 0, 1, 0.333, 0.621]	[0, 0, 0, 1, 0.333, 0.621]
{B,D,E}	[1.0, 1.0, 1.0, 1, 0.167, 0.621]	[1.414, 1.333, 1.5, 1, 0.167, 0.621]

8.5 结果解释

这个小实验里最关键的现象是：

- `{A,C,D}` 在 `n=3` 下提升最大；
- 因为它直接受益于 4 阶超边 `{A,B,C,D}` 被分解为多个 3 阶子超边；
- 结构较弱的候选集合，如 `{A,E,F}`，几乎不会因此变强。

这说明：

更高阶结构经过逐步降阶后，会对真正嵌在高阶结构中的候选集合提供更强的结构支持。

这就是论文大实验中“随着保留更多高阶信息，AUC 往往上升”的微观机制。

8.6 玩具实验完整代码

文档配套的完整脚本见：

- toy_n_reduction_experiment.py

最小代码如下：

```
from itertools import combinations
from math import log, exp

nodes = ['A', 'B', 'C', 'D', 'E', 'F']

original_hyperedges = [
    ('A', 'B', 'C'),
    ('A', 'B', 'D'),
    ('A', 'B', 'C', 'D'),
    ('B', 'C', 'E'),
    ('C', 'D', 'F'),
]

candidates = [
    ('A', 'C', 'D'),
    ('A', 'C', 'E'),
    ('A', 'D', 'F'),
    ('A', 'E', 'F'),
    ('B', 'D', 'E'),
]

def reduce_hyperedges(hyperedges, n):
    reduced = []
    for edge in hyperedges:
        if len(edge) ≤ n:
            reduced.append(tuple(sorted(edge)))
        else:
            for combo in combinations(edge, n):
                reduced.append(tuple(sorted(combo)))
    seen = set()
    result = []
    for edge in reduced:
        if edge not in seen:
            seen.add(edge)
            result.append(edge)
    return result

def build_pair_graph(hyperedges):
    pair_weights = {}
    neighbors = {v: set() for v in nodes}
    for edge in hyperedges:
        for u, v in combinations(edge, 2):
            a, b = sorted((u, v))
            pair_weights[(a, b)] = pair_weights.get((a, b), 0) + 1
            neighbors[u].add(v)
            neighbors[v].add(u)
    return pair_weights, neighbors

def extract_features(candidate, hyperedges):
    pair_weights, neighbors = build_pair_graph(hyperedges)
    weights = []
    for u, v in combinations(candidate, 2):
        a, b = sorted((u, v))
        if (a, b) in pair_weights:
            weights.append(pair_weights[(a, b)])
    if weights:
        gm = exp(sum(log(w) for w in weights) / len(weights))
```

```
        hm = len(weights) / sum(1 / w for w in weights)
        am = sum(weights) / len(weights)
    else:
        gm = hm = am = 0
    common_neighbors = set(neighbors[candidate[0]])
    for v in candidate[1:]:
        common_neighbors &= neighbors[v]
    union_neighbors = set()
    for v in candidate:
        union_neighbors |= neighbors[v]
    cn = len(common_neighbors)
    jc = cn / len(union_neighbors) if union_neighbors else 0
    aa = sum(1 / log(len(neighbors[v])) for v in common_neighbors if len(neighbors[v]) > 1)
    return {"GM": round(gm, 3), "HM": round(hm, 3), "AM": round(am, 3),
            "CN": cn, "JC": round(jc, 3), "AA": round(aa, 3)}
```

---

9. 实验设计与方法细节

9.1 数据集

作者使用 12 个真实超图数据集，覆盖：

- 社交通信
- 药物-疾病关系
- 生化代谢
- 生态寄生
- 学术合作

为控制组合爆炸，论文只考虑超边阶数 `k ≤ 10`。

9.2 Table 1 数据集概览

数据集	类型/领域	节点数 N	超边数 M	平均超边阶
email-Eu	社会通信	530	1119	2.51
email-Enron	社会通信	143	1466	2.95
threads-math-sx	社会通信	2563	2918	2.50
DAWN	药物-疾病关系	895	7728	3.15
NDC-classes	药物-疾病关系	1140	940	4.43
NDC-substances	药物-疾病关系	2647	4606	4.87
iAF1260b	生化代谢	1665	2021	4.11
iAF987	生化代谢	1098	1162	4.47
iCN900	生化代谢	879	1086	4.45
Nematode	生态寄生	4570	5695	3.55
Pubmed	学术合作	3772	7034	3.55
DBLP	学术合作	15623	18383	2.79

9.3 数据划分与负采样

- 训练集 / 测试集 = `8 : 2`
- 训练阶段使用 `5-fold cross-validation`
- 做 `10` 次独立实验取平均

由于候选超边数量极大，作者进行了负采样。

论文正文描述的思路是：

- 对每个正超边随机删去一个节点
- 再从邻居中替换进一个节点
- 如果新集合不是原始超边，就作为负样本

但代码仓库中的实现更接近：

- 在投影得到的图中采样 clique 作为负样本

因此，严格复现时需要特别注意：

论文描述与仓库实现的负采样细节并不完全一致。

9.4 六个特征

作者采用 6 个可解释特征，而不是复杂深度模型。

把候选节点集合当作整体看

- `CN`：共同邻居数
- `JC`：Jaccard 系数
- `AA`：Adamic-Adar 指数

把候选节点集合拆成多个节点对看

- `GM`：权重几何平均
- `HM`：权重调和平均
- `AM`：权重算术平均

这里的节点对权重 `W_uv` 表示两个节点共同出现在多少条超边中。

9.5 分类器

作者采用：

- `Logistic Regression`

原因是：

论文的重点是比较结构表示方式，而不是让复杂模型掩盖结构差异。

---

10. 论文结果解读

10.1 结果一：AUC 随 `n` 增大总体上升，但边际收益递减

主要现象是：

- 当 `n` 从较小值增大时，AUC 往往明显提升；
- 再继续增大时，收益会逐渐变缓，进入平台区。

这意味着：

1. 更多高阶信息通常是有价值的；
2. 但并不是“阶数越高越好”；
3. 高阶信息存在边际收益递减。

论文中很适合汇报的例子是：

- 当预测三阶超边时，`n` 从 3 增加到 7，
- `NDC-classes` 的 AUC 从约 `0.46` 提升到 `0.72`
- `NDC-substances` 的 AUC 从约 `0.68` 提升到 `0.87`

这说明在某些药物相关网络中，高阶信息对超边预测极其关键。

10.2 结果二：不同网络的收益差异明显

论文并没有得出“高阶一定重要”的结论，而是指出：

- 一些网络对高阶结构非常敏感；
- 另一些网络，即便加入更高阶信息，收益也不明显。

因此，是否值得做高阶建模，必须结合具体网络和具体任务来判断。

10.3 结果三：n-reduced 显著优于 n-projected

在统一的 6 个特征 + Logistic Regression 设置下，论文给出：

- `n-projected`：最佳 AUC 约 `0.74`，平均约 `0.62`
- `n-reduced`：最佳 AUC 约 `0.94`，平均约 `0.83`

这说明：

如果必须对高阶结构做简化，那么 `n-reduced` 比 `n-projected` 更能保留对任务有用的结构信息。

10.4 结果四：结论具有一定稳健性

作者还使用了多种超边预测模型进行对照，例如：

- HPRA
- NHP
- NHNE

补充材料显示总体结论保持一致。

这意味着：

本文的核心发现不只是一个特定模型设置下的偶然现象。

10.5 关键图示

图 1 与表 1

Table 1 | The statistics of the twelve real hypergraphs

Dataset	Type	N	M	$\langle k \rangle$	Reference
email-Eu	Social communication	530	1119	2.51	29,30,31
email-Enron	Social communication	143	1466	2.93	29
threads-math-sx	Social communication	2563	2918	2.50	29
DAWN	Drug-disease relationship	895	7728	3.13	29
NDC-classes	Drug-disease relationship	1140	940	4.43	29
NDC-substances	Drug-disease relationship	2647	4606	4.87	29
iAF1260b	Biochemical metabolism	1668	2021	4.11	32
iAF987	Biochemical metabolism	1098	1162	4.47	32
iCN900	Biochemical metabolism	879	1036	4.45	32
Nematode	Ecology parasitology	4570	5695	3.33	33,34
Pubmed	Academic collaboration	3772	7034	3.33	35
DBLP	Academic collaboration	13523	18383	2.79	36

Here N is the number of nodes, M is the number of hyperedges, and $\langle k \rangle$ is the average order of hyperedges.

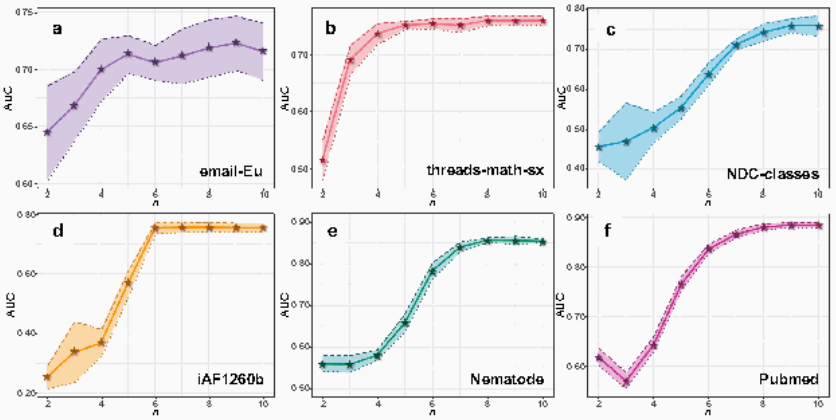


Fig. 1 | The change of the average AUC (area under the ROC curve) with increasing n in the n-reduced graph. a–f show the results corresponding to datasets email-Eu, threads-math-sx, NDC-classes, iAF1260b, Nematode, and Pubmed, respectively. The solid line represents the average AUC over ten independent experiments, and the shaded area indicates the range of the average AUC plus and minus the standard deviation.

In V_n^ℓ, node v_n and node u_n are connected if they satisfy the following two conditions: (1) v_n and u_n differ by exactly one element among the $n-1$ elements each contains, i.e., $|u_n \cup v_n| = n$; (2) the union of v_n and u_n is fully contained in at least one hyperedge in E. Accordingly,

$$E_n^\ell := \{(u_n, v_n) : u_n \in V_n^\ell,\ v_n \in V_n^\ell,\ |u_n \cup v_n| = n,\ \exists e \in E, s.t., u_n \cup v_n \subseteq e\}. \tag{2}$$

For a given n, the graph $G^\ell(n)$ used for hyperedge prediction is constructed as a direct aggregation from the 2-projected graph up to the n-projected graph, denoted as $G^\ell(n) := \{G_1^\ell, G_2^\ell, \cdots, G_n^\ell\}$. It is worth noting that for any $n \geq 2$, $G^\ell(n)$ contains only pairwise interactions and its complexity primarily comes from the quantity and heterogeneity of nodes. As each G_n^ℓ represents a union of sets rather than multisets, any duplicate instances of a hyperedge e will be excluded. Although the n-projected and n-reduced graphs are conceptually similar in that they both decompose higher-order interactions, they differ fundamentally in how this decomposition is performed. Therefore, we are particularly interested in determining which method preserves more useful structural information. A natural assumption is that the one retaining more useful information will yield better performance in hyperedge prediction. For comparison, we use the same six features and logistic regression model to predict hyperedges in the n-projected graph, following the method proposed by ref. 40. Based on an edge's attribute in $G^\ell(n)$, we can identify its order in the original hypergraph. For instance, an edge between two sets of four nodes corresponds to a five-order hyperedge in the original hypergraph. Figures 3 and S6 (see Supplementary Note 4) compare the predictive performance of these two methods across hyperedge orders ranging from 3 to 6, with $n = 7$. The results show that the n-reduced method significantly outperforms the n-projected method across all

n-reduced 与 n-projected 对比图

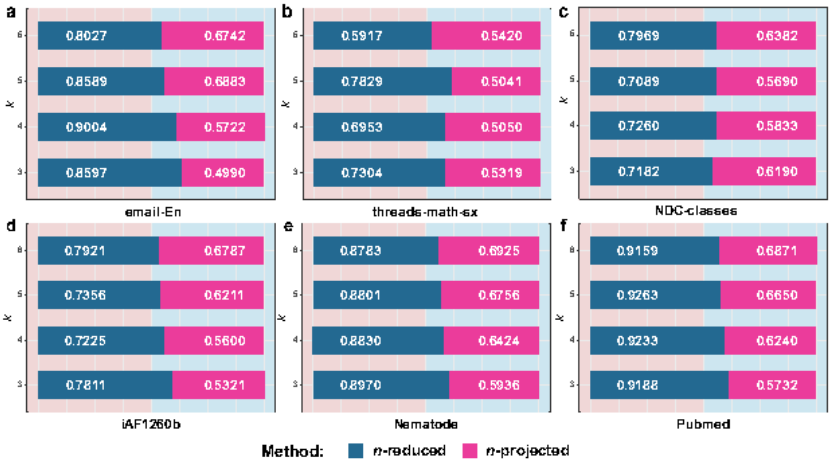


Fig. 3 | **Comparison of the AUC (area under the ROC curve) values between n-reduced and n-projected operators for predicting *k*-order hyperedges.** a–f show the results corresponding to datasets email-Eu, threads-math-sx, NDC-classes, iAF1260b, Nematode, and Pubmed, respectively. Here, n is set to 7, and all results are averaged over ten independent experiments. The y-axis represents the order k of the predicted hyperedges. The dark blue bars denote the average AUC for the n-reduced method, while the magenta bars represent the average AUC for the n-projected method. To provide a clear comparison of prediction accuracy between the two methods, the length of each bar is scaled according to the AUC value of each method as a proportion of the sum of AUC values of both methods. For instance, if the AUC for the n-reduced method is 0.8 and the AUC for the n-projected method is 0.7, the lengths of the dark blue and magenta bars would be 8/15 and 7/15, respectively.

nodes unchanged. Although these two methods share the same starting point, the n-reduced graph may be more suitable as an analyzing tool, because researchers seem to be less inclined to deal with heterogeneous nodes. Otherwise, bipartite graphs would be more popular than hypergraphs for representing higher-order interactions.

In this study, we compare the n-projected and n-reduced methods by employing link prediction as an entry point to quantify the influence of higher-order interactions. Link prediction is chosen because it serves as a more fundamental task compared to the analysis of specific networked dynamics. Although our method has yielded promising results in hyperedge prediction, its applicability to other tasks–such as community detection and the identification of critical nodes–remains to be further exploration. From the experiment in this study, there are two main conclusions: First, higher-order interactions generally have a significant and positive effect on predictive accuracy, although their effectiveness may vary across different networks and is not universally guaranteed. Second, compared to the n-projected method, the n-reduced method proposed in the study retains more higher-order information. This is evidenced by the fact that the n-reduced method achieves a higher accuracy in hyperedge prediction under the same conditions.

In conclusion, we offer three specific suggestions based on our findings. First, our results show that higher-order interactions have a significant and positive impact on hyperedge prediction. Therefore, when computationally feasible, researchers are encouraged to prioritize the use of hypergraphs to represent higher-order interactions, rather than reducing them to pairwise interactions. Second, we observed that the influence of higher-order interactions varies significantly across different real-world networks. Therefore, in order to draw more comprehensive and reliable conclusions, further analysis should be conducted on domain-specific and topology-specific hypergraphs. This may even include the distinction of which hyperedges bring richer information in a given hypergraph[55]. In addition, we propose a toy model to investigate whether a small fraction of all possible nested lower-order hyperedges can effectively retain the structural information of higher-order interactions. As shown in Supplementary Note 6, retaining only 20% of reduced hyperedges–specifically those involving large-degree nodes–can achieve very close prediction accuracy to the original n-reduction method. Finally, as the n-reduced operator can effectively retain higher-order interaction information, we recommend researchers use the n-reduced graphs as an analytical tool to quantify the role of higher-order interactions on specific dynamics (e.g., propagation and synchronization)[56–59] and other graph mining tasks (e.g., critical nodes identification and community detection)[60–62].

As with any research, this study has its limitations. The n-reduced method is heuristic in nature and lacks a rigorous mathematical or physical theory to quantify how much information it remains. Therefore, an important yet challenging direction for future study is how to design a rigorous theory. Maybe one can first consider a statistical theory on hypergraph configuration model[63]. Benson et al.[23] attempted to use lower-order interaction data to predict the closure of simplicial complexes, and thus, another issue for future study is to extend the current stepwise decomposition method for simplicial complexes. Additionally, future investigations could benefit from exploring diffusion models or statistics-driven approaches. These methods may provide deeper theoretical insights into the effectiveness of hypergraph-based analyses, thereby addressing some of the current limitations.

Methods

The *n*-reduced graph

In order to analyze the role of all hyperedges with orders larger than a threshold order, we propose the following method, referred to as the n-reduced graph. Given a hypergraph $G(V, E)$, its corresponding n-reduced

n-reduced 构造示意图

https://doi.org/10.1038/s42005-025-02157-3

graph, denoted by $G_n(V, E_n)$ is defined as

$$E_n := \{e_\alpha : e_\alpha \in E, |e_\alpha| \le n\} \cup \{e : e_\beta \in E, |e_\beta| > n, e \subseteq e_\beta, |e| = n\}. \quad (3)$$

As defined above, the hyperedge set E_n comprises two parts: (1) all hyperedges in G with $k_\alpha \le n$ are retained in E_n; (2) the hyperedges with $k_\alpha > n$ are decomposed into $\binom{k_\alpha}{n}$ hyperedges, each of which has an order of n. Note that when $n = 2$, G_n is equivalent to the unweighted pairwise projection of G; and when $n = \max_\alpha\{k_\alpha\}$, G_n is equivalent to the original hypergraph G. Unlike the n-projected graph, the n-reduced graph retains the node set and all hyperedges with order no greater than n, while decomposing hyperedges of order greater than n using sets of hyperedges with order n. In summary, the n-reduced graph is an approximated representation of the original hypergraph that consists of hyperedges of orders no greater than n, with the goal of minimizing information loss, which is particularly suitable for evaluating the role of hyperedges with orders larger than a threshold. Figure 4 illustrates the construction process of a three-reduced graph. This hypergraph consists of eight nodes and seven hyperedges, with the orders of six hyperedges (e_1, e_2, e_3, e_4, e_5, and e_6) not exceeding 3. These hyperedges are directly retained in the 3-reduced graph. For the hyperedge e_7 with $k_\alpha > 3$, we generate all three-order hyperedges by traversing all triples from the set of nodes in the hyperedge e_7, which are subsequently added to the three-reduced graph.

Features and classifier

This study applies three local similarity features and three weighted features to evaluate the likelihood of a set of nodes to form a hyperedge. The former measures whether the nodes in the set are all adjacent to some other nodes (two nodes are adjacent if they appear together in at least one hyperedge), while the latter measures the closeness between pairs of nodes. In brief, the former treats the candidate node set as a whole, whereas the latter views it as the sum of multiple pairwise relationships.

In this study, local similarity features are direct extensions of classic similarity indices used in link prediction[64–69]. (1) Common neighbor index: The common neighbors of a hyperedge e are the common neighbors of all nodes in e, that is

$$CN(e) = |\bigcap_{v \in e} N(v)|, \quad (4)$$

where $N(v)$ represents the neighbor set of the node v. (2) Jaccard coefficient: The Jaccard coefficient is a normalization index that divides the number of common neighbors among all nodes in hyperedge e by the total number of neighbors of any nodes in e:

$$JC(e) = \frac{|\bigcap_{v \in e} N(v)|}{|\bigcup_{v \in e} N(v)|}. \quad (5)$$

(3) Adamic–Adar index: The Adamic–Adar index penalizes high-degree nodes by dividing the degrees of common neighbors, as

$$AA(e) = \sum_{v \in \bigcap_{v \in e} N(v)} \frac{1}{\log d_i}. \quad (6)$$

Weighted features measure the closeness among the candidates node set by using different averages of the weights of direct connections between node pairs. Let W_{uv} denote the weight of the connection between nodes u and v, which is defined as the number of hyperedges containing both u and v. Clearly, if u and v are adjacent, then $W_{uv} > 0$, otherwise, $W_{uv} = 0$. For any candidate node set e, let E_e represent the set of pairs of adjacent nodes in e, that is

$$E_e := \{(u, v) : u \in e, v \in e, W_{uv} > 0\}. \quad (7)$$

Note that $u \in e$ and $v \in e$ do not necessarily imply $W_{uv} > 0$, as e may not be a hyperedge in the original hypergraph. Accordingly, we can calculate the average weight of all adjacent node pairs in e using three different methods. (4) Geometric mean:

$$GM(e) = \left(\prod_{(u,v) \in E_e} W_{uv}\right)^{\frac{1}{|E_e|}}. \quad (8)$$

(5) Harmonic mean:

$$HM(e) = \frac{|E_e|}{\sum_{(u,v) \in E_e} W_{uv}^{-1}}. \quad (9)$$

(6) Arithmetic mean:

$$AM(e) = \frac{\sum_{(u,v) \in E_e} W_{uv}}{|E_e|}. \quad (10)$$

This study integrates the six features into a hyperedge feature vector and utilizes the Logistic regression to train the model[70]. That is, the feature vector of a candidate node set e can be represented as a linear combination of

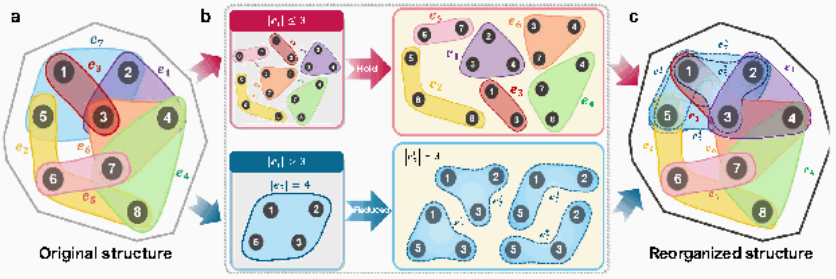
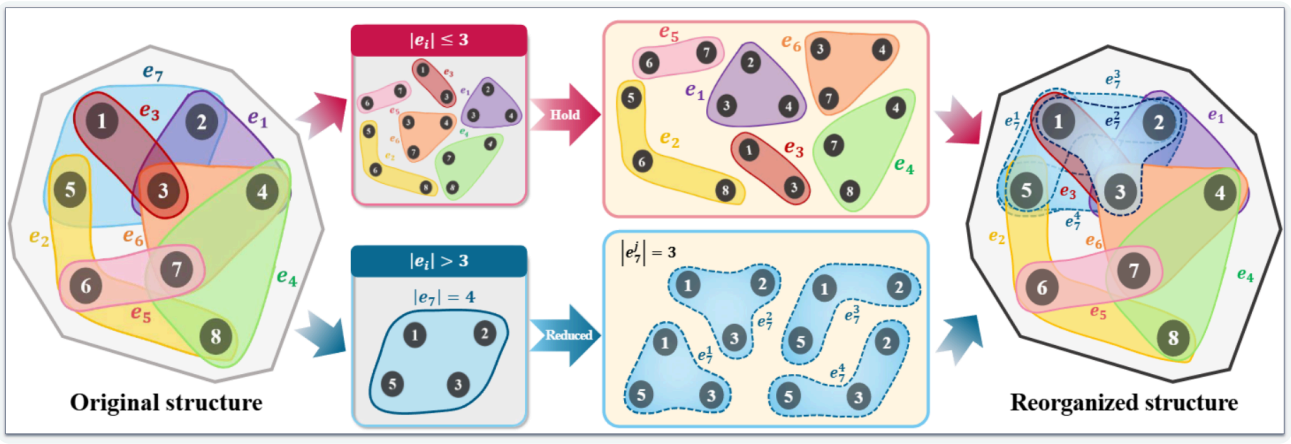


Fig. 4 | The process to construct the n-reduced graph. a shows the original structure, b illustrates the 3-reduction process, and c displays the reorganized structure. This hypergraph contains eight nodes and seven hyperedges. Among them, the orders of e_2, e_5, and e_3 are 2, the orders of e_1, e_6 and e_4 are 3, and the order of e_7 is 4. When $n = 3$, hyperedge e_7 is decomposed into all combinations of its nodes, say $\{v_1, v_2, v_3\}$, $\{v_1, v_2, v_4\}$, $\{v_1, v_3, v_4\}$, and $\{v_2, v_3, v_4\}$.

仓库中的示意图



11. email-Enron 数据集结构说明

这是你已经实际复现并得到结果的数据集，因此很适合单独讲清楚。

11.1 数据集含义

根据作者给出的说明文件，`email-Enron` 数据集来自 Enron 邮件数据集。

其含义是：

- 每条超边对应一封邮件；
- 超边中的节点是该邮件涉及的 Enron 员工；
- 包括发送者和接收者；
- 只保留 Enron 员工作为节点；
- 只保留“发送者是 Enron 员工，且至少一个接收者也是 Enron 员工”的邮件。

因此在这个数据集中：

- 节点 = Enron 员工
- 超边 = 一封邮件所涉及的员工集合
- 时间戳 = 该邮件发生的时间

11.2 文件结构

核心文件有 3 个：

1. `nverts.txt`
2. `simplices.txt`
3. `times.txt`

辅助映射文件：

4. `email-Enron-node-labels.txt`

11.3 每个文件表示什么

`nverts.txt`

每行一个整数，表示对应那条超边包含多少个节点。

例如开头可能是：

```
2
3
2
2
```

表示：

- 第 1 条超边有 2 个节点
- 第 2 条超边有 3 个节点
- 第 3 条超边有 2 个节点

该文件共有 `10883` 行，对应 `10883` 条时间戳事件记录。

`simplices.txt`

把所有超边中的节点编号顺序拼接在一起。

例如：

```
4
1
117
129
1
51
1
```

如果 `nverts.txt` 前几项是 `2, 3, 2`，则表示：

- 第 1 条超边：`{4,1}`
- 第 2 条超边：`{117,129,1}`
- 第 3 条超边：`{51,1}`

这个文件一共有 `26841` 行。

`times.txt`

每行一个整数，对应每条超边的时间戳，与 `nverts.txt` 一一对应。

例如：

```
63083183340000
63084674640000
63085775220000
```

`email-Enron-node-labels.txt`

给出节点编号和真实邮箱地址之间的映射。

例如：

```
1 phillip.allen@enron.com
2 john.arnold@enron.com
3 harry.arora@enron.com
```

该文件共有 `148` 行，说明涉及 `148` 个 Enron 员工节点。

11.4 为什么它适合这篇论文

这个数据集天然适合超图建模，因为一封邮件本身就是一个“多人共同交互事件”。

如果只用普通图表示，就只能把一封多人邮件拆成很多两两边，会丢失“这是同一封邮件共同参与”的整体语义。

---

12. 你的 email-Enron 复现结果与论文结果对比

你目前复现出的 AUC 结果为：

n	你的 AUC
2	0.547949
3	0.642513
4	0.705037
5	0.687511
6	0.691622

n	你的 AUC
7	0.695916
8	0.689073
9	0.690892
10	0.716010

论文源数据 `Figure1_S3/email-Enron.csv` 为：

n	论文均值	论文标准差
2	0.604484	0.023981
3	0.673693	0.017388
4	0.685035	0.018055
5	0.696337	0.023650
6	0.688510	0.013022
7	0.684893	0.019088
8	0.704088	0.018169
9	0.709277	0.008836
10	0.695334	0.017691

12.1 逐点对比

n	你的结果	论文均值	差值
2	0.547949	0.604484	-0.056534
3	0.642513	0.673693	-0.031181
4	0.705037	0.685035	+0.020002
5	0.687511	0.696337	-0.008826
6	0.691622	0.688510	+0.003113
7	0.695916	0.684893	+0.011022
8	0.689073	0.704088	-0.015014
9	0.690892	0.709277	-0.018385
10	0.716010	0.695334	+0.020676

12.2 如何解释这个结果

这个结果说明：

1. **总体趋势是一致的**
   你的结果和论文都呈现：
   - `n=2` 较低
   - `n=3` 后明显上升
   - 后续在 `0.68 ~ 0.72` 附近波动

2. **单点数值不完全一致是正常的**
   因为你当前跑的是单次实验，而论文给的是多次独立实验的平均值。
3. **你已经基本复现了论文的核心现象**
   即：
   - 随着保留更多高阶信息，预测性能总体提升；
   - 之后进入平台和波动区。

12.3 为什么会有偏差

主要原因可能有：

1. 你当前跑的是单次实验，而论文做了 10 次独立实验取平均；
2. 训练集 / 测试集随机划分不同；
3. 负采样带有随机性；
4. 环境与依赖版本也可能带来轻微差异。

因此，目前更准确的说法是：

- 你已经完成了**趋势级复现**
- 但还不是**数值级严格复现**

12.4 可以怎么向导师汇报

可以直接这样说：

> 我已经在 `email-Enron` 数据集上复现了论文的主实验趋势。虽然单次实验得到的具体 AUC 数值和论文中的多次实验均值不完全一致，但整体趋势是一致的：当 `n` 从 2 增大时，预测性能明显提升，随后在较高阶区域进入波动平台。这说明论文关于“保留更多高阶信息通常有助于超边预测，但增益会趋于平缓”的核心结论是可以复现的。

---

13. 论文的优势与局限

13.1 优势

1. **可控、可解释**
   单参数 `n` 提供了清晰的信息-复杂度调节旋钮。
2. **保持原始节点集合不变**
   比 `n-projected` 更贴近原始超图语义。
3. **适合做高阶信息贡献分析**
   能较自然地回答“保留到几阶才够”。
4. **具有一定复现性**
   作者公开了代码和数据。

13.2 局限

1. **组合爆炸仍然存在**
   对每条高阶超边生成 `C(k,n)` 个子超边，会快速膨胀。
2. **缺少严格理论刻画**
   目前仍是启发式近似，缺乏信息保真度的严格理论分析。
3. **任务覆盖偏单一**
   核心实证主要围绕超边预测。
4. **简单模型有利于解释，但也限制了讨论范围**
   论文没有充分展开更复杂 HNN 模型下高阶信息的作用方式。

---

14. 面向超图研究的应用与扩展设想

14.1 方向一：作为超图学习的降阶预处理

思路：

- 用 `n-reduced` 把超图压到某个最大阶数；
- 再喂给超图神经网络或其他模型；
- 比较性能与计算成本的折中。

可以研究：

1. 不同 `n` 下性能如何变化；
2. 训练时间和显存如何变化；
3. 是否存在任务最优的 `n`。

14.2 方向二：做稀疏型/学习型 n-reduction

思路：

- 并不是每个 `n` 元子超边都同等重要；
- 可以只保留最有价值的一部分；
- 从而在更少结构代价下保留主要信息。

可做方向：

1. 基于节点度的启发式保留；
2. 基于学习模型对子超边打分；
3. 在固定预算下做最优选择。

14.3 方向三：构建高阶价值剖面

思路：

- 不只在超边预测上看 `AUC(n)`；
- 还可以在传播、同步、社区检测等任务上看任务输出如何随 `n` 变化；
- 从而得到一个“到几阶才够”的任务剖面曲线。

---

15. 10 到 15 分钟导师汇报建议结构

Slide 1：研究动机

- 现实系统中存在大量高阶相互作用
- 超图更真实，但计算更复杂
- 问题：高阶信息到底值不值得保留

Slide 2：相关工作脉络

- 高阶网络综述
- 高阶链接/超边预测基准
- `n-projected`
- 本文 `n-reduced`

Slide 3：问题设定

- 用超边预测作为统一量尺
- AUC 作为性能指标
- 通过控制 `n` 来观察高阶信息贡献

Slide 4：方法对比

- 团扩展
- 星拓展
- `n-projected`
- `n-reduced`

Slide 5：n-reduced 的定义与例子

- 保留不高于 `n` 阶超边
- 高于 `n` 阶的超边拆成全部 `n` 元子超边
- 用小例子解释

Slide 6：实验流程

- 数据划分
- 负采样
- 6 个特征
- Logistic Regression
- AUC 评估

Slide 7：数据集

- 12 个真实超图
- 覆盖社交、药物、代谢、生态、合作网络

Slide 8：结果一

- AUC 随 `n` 提升
- 但边际收益递减
- 不同网络差异显著

Slide 9：结果二

- `n-reduced` 显著优于 `n-projected`
- 说明其更适合作为高阶信息贡献分析工具

Slide 10：局限与启发

- 启发式近似
- 组合爆炸
- 任务覆盖有限
- 可扩展到 HNN 加速、稀疏降阶、高阶价值剖面

---

16. 可能被导师追问的问题

Q1：n-reduced 会不会制造原本不存在的超边？

会生成派生的 `n` 阶子超边，但它的目标不是保持事件语义一一对应，而是作为分析工具尽量保留高阶结构中的信息。

Q2：为什么不用更复杂的深度模型？

作者刻意使用简单可解释模型来控制变量，以突出结构表示本身的差异，而不是让复杂模型掩盖方法对比。

Q3：怎么选 `n`？

论文当前的做法是扫描不同 `n`，观察 AUC 曲线及边际收益变化；目前还没有成熟理论自动给出最优 `n`。

Q4：复杂度瓶颈在哪里？

主要在于一条高阶超边需要展开成大量 `n` 元子超边，即组合爆炸问题。

Q5：与 pairwise 投影的关系是什么？

当 `n = 2` 时，`n-reduced` 就退化到 pairwise 层面，因此它实际上提供了一个从 pairwise 到原始超图的连续谱系。

Q6：为什么不同网络的高阶收益差异这么大？

因为有些网络的高阶结构可以被低阶较好近似，而另一些网络中的高阶交互本身不可约，因此更依赖高阶建模。

Q7：负采样会不会影响结果？

会有影响，因此严格复现时需要关注负采样策略的差异。作者通过平衡采样和多次重复实验来增强稳健性，但这仍是一个值得做敏感性分析的点。

---

17. 作者仓库中的实验主线

结合论文和代码，可以把作者仓库中的实验主线概括为：

1. 从数据文件中读取超边；
2. 构造由超边投影得到的普通图与节点对权重矩阵；
3. 随机划分训练集和测试集；
4. 对训练集做 `n-reduction`；
5. 生成负样本；
6. 提取 6 个结构特征；
7. 使用逻辑回归训练模型；
8. 用 5 折交叉验证选择参数；
9. 在测试集上计算 AUC；
10. 对不同 `n` 重复实验并输出结果。

核心代码文件：

- main.py
- data_reader.py
- hyperedge_model.py
- feature_extractor.py
- model_trainer.py

---

18. 服务器复现项目的基本步骤

18.1 克隆项目

```
git clone https://github.com/jackyjh/n-reduction-graph.git
cd n-reduction-graph
```

18.2 创建环境

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install numpy networkx scikit-learn
```

18.3 注意 Linux 下目录大小写

仓库顶层目录通常是 `Dataset`，但代码里默认写的是 `dataset`。

Linux 区分大小写，因此建议：

```
mv Dataset dataset
```

或者修改源码中的路径。

18.4 运行主脚本

```
cd src
python main.py
```

18.5 需要注意的点

1. 仓库默认配置更像测试版，不一定就是论文全量实验；
2. 如果要更接近论文，需要把数据集列表和实验次数改成完整设置；
3. 负采样实现与论文正文存在细节差异，严格复现时要记录这一点。

---

19. 总结陈述

如果要用一段话向导师总结这篇论文，可以这样说：

> 这篇论文最重要的贡献，不是单纯提高了超边预测精度，而是提出了一个 `n-reduced` 的逐步降阶框架，使我们可以在尽量少损失信息的条件下，定量分析高阶相互作用的真实贡献。实验表明，高阶信息在不同类型超图中的作用并不一致：在部分药物、生态和学术合作网络中非常关键，但在某些社交网络中增益有限。因此，这项工作对超图研究的意义在于，它提供了一个判断“是否值得使用高阶建模”的分析工具，并可以进一步应用到超图神经网络、社区发现、关键节点识别和传播动力学等方向。

---

20. 参考材料

- 论文 PDF：`Unveiling the role of higher-order interactions via stepwise reduction.pdf`
- 论文 DOI：https://doi.org/10.1038/s42005-025-02157-3

- 代码仓库：https://github.com/jackyjh/n-reduction-graph
- 配套玩具实验脚本：toy_n_reduction_experiment.py